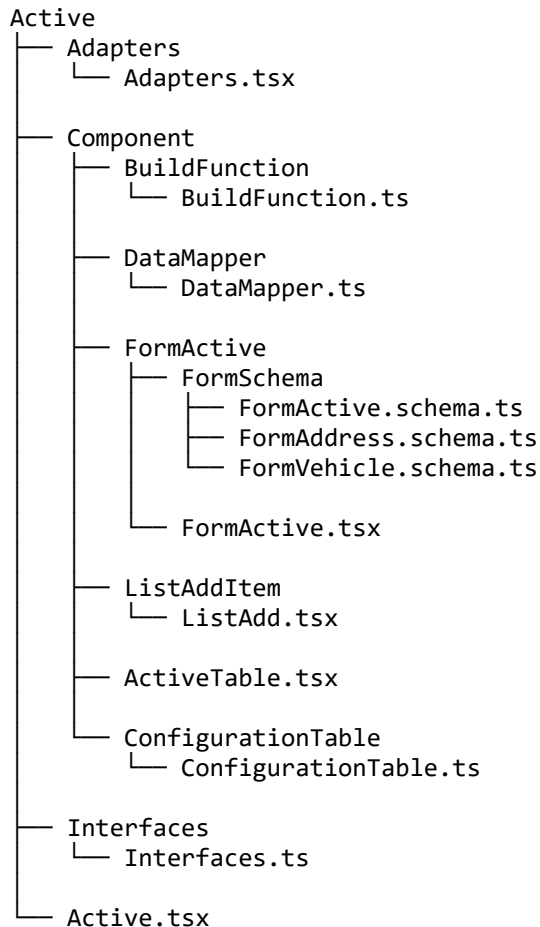


Módulo Active

O **Módulo Active** é responsável pela gestão de ativos dentro do sistema, concentrando componentes de interface, regras de montagem de dados, mapeamentos e contratos de tipagem.

Estrutura do Módulo



Descrição Arquitetural

Active.tsx

Arquivo principal do módulo, responsável por orquestrar os componentes, controlar o fluxo da tela e integrar as funcionalidades de listagem, formulário e configuração.

Adapters/Adapters.tsx

Camada de adaptação entre os dados vindos da API e o formato utilizado pela aplicação.

- Normalizar dados externos
- Converter estruturas de resposta
- Preparar dados para uso nos componentes internos
- Dentro da aplicação estamos usando uma arquitetura Dry (Dont Repeat yor'self)

```

import { fetchDataFull } from "../../Util/Util";
import { FetchConfig } from "../Interfaces/Interfaces";

function fetchGet({ pathFile, urlComplement = "" }: FetchConfig) {
    return fetchDataFull({
        method: "GET",
        params: null,
        pathFile,
        urlComplement
    });
}

function fetchPost({ pathFile, params = {}, urlComplement }: FetchConfig) {
    return fetchDataFull({
        method: "POST",
        params,
        pathFile,
        urlComplement
    });
}

function getAll(pathFile: string) {
    return fetchGet({
        pathFile,
        urlComplement: "&all=1"
    });
}

function getByParam(pathFile: string, param: string, value: string) {
    return fetchGet({
        pathFile,
        urlComplement: `&${param}=${value}`
    });
}

function postToApi(pathFile: string, params: {}) {
    return fetchPost({
        params,
        pathFile,
        urlComplement: "&v2=1&smart=ON",
    })
}

export function ActivePostData(params: {}) {
    return postToApi("GAPP_V2/Active.php", params);
}

export function ActiveData() {
    return getAll("GAPP/Active.php");
}

export function ActiveCompanyData() {
    return getAll("GAPP/Company.php");
}

export function ActiveTypeData() {
    return getAll("GAPP/ActiveType.php");
}

export function ActiveVehicleData(active_id: string) {
    return getByParam("GAPP/Vehicle.php", "active_id_fk", active_id);
}

export function ActiveUnitsData() {

```

```

    return getAll("GAPP/Units.php");
}

export function ActiveDepartamentData() {
    return getAll("GAPP/Departament.php");
}

export function ActiveDriverData() {
    return getAll("GAPP/Driver.php");
}

export function ActiveFuelData() {
    return getAll("GAPP/Fuel.php");
}

export function ActiveTypeFuelData() {
    return getAll("GAPP/TypeFuel.php");
}

```

◆ Interfaces/Interfaces.ts

Define os contratos de tipagem do módulo.

- Centralizar interfaces e tipos TypeScript
- Garantir padronização dos modelos de dados
- Aumentar a segurança e previsibilidade do código

◆ Component/BuildFunction/BuildFunction.ts

Responsável por funções de construção e preparação de estruturas de dados.

```

export function buildOptions(apiData?: FormActiveProps["apiData"]) {
    return {
        company: apiData?.company?.map((c) => ({
            label: c.corporate_name,
            value: String(c.comp_id),
        })) || [],

        departament: apiData?.departament?.map((d) => ({
            label: `${d.fantasy_name} > ${d.unit_name} > ${d.dep_name}`,
            value: String(d.dep_id),
        })) || [],

        unit: apiData?.unit?.map((u) => ({
            label: u.unit_name,
            value: String(u.unit_id),
        })) || [],

        driver: apiData?.driver?.map((d) => ({
            label: d.name,
            value: String(d.driver_id),
        })) || [],

        fuel: apiData?.fuelType?.map((f) => ({
            label: f.description,
            value: String(f.id_fuel_type),
        })) || [],
    };
}

```

}

- Monta os options para os selects
- Preparar dados para consumo tudo configurado
- Estrutura facil de entender e passar para os formularios consumirem consumirem

◆ **Component/DataMapper/DataMapper.ts**

Camada de mapeamento de dados entre:

- Estrutura o payload
- Diminui a complexidade de entender o que os dados estão fazendo.
- Ajuda a fazer a manutenção

Ajuda a manter o desacoplamento entre backend e frontend.

◆ **Component/FormActive/FormActive.tsx**

Componente responsável pela tela de cadastro/edição do ativo.

- Estado do formulário
- Validações
- Submissão de dados
- Integração com schemas e mapeadores

FormSchema

Contém os schemas de validação e definição estrutural dos formulários:

- FormActive.schema.ts → Regras do formulário principal do ativo
- FormAddress.schema.ts → Regras do formulário de endereço
- FormVehicle.schema.ts → Regras do formulário de veículo

Essa separação melhora:

- Organização
- Reuso
- Manutenibilidade
- Clareza das regras de negócio

◆ **Component/ListAddItem/ListAdd.tsx**

Componente responsável por gerenciar listas dinâmicas de itens, permitindo inclusão e manipulação de registros associados ao ativo.

◆ **Component/ActiveTable.tsx**

Responsável pela exibição tabular dos ativos.

- Renderização da listagem dinamica
- Exibição de colunas e ações
- Tem uma manutenção facil e precisa.

◆ Component/ConfigurationTable/ConfigurationTable.ts

Define a configuração estrutural da tabela, como:

- Colunas
- Formatação de dados
- Regras de exibição
- Comportamento visual



Visão Arquitetural Resumida

- **Active.tsx** → Orquestrador do módulo
- **Adapters / DataMapper / BuildFunction** → Camada de transformação e preparação de dados
- **Interfaces** → Contratos e tipagem
- **FormActive + Schemas** → Entrada e validação de dados
- **ActiveTable + ConfigurationTable** → Exibição e configuração de dados
- **ListAddItem** → Gestão de listas dinâmicas



Contratos e Modelos de Dados (Interfaces)

Esta seção documenta os principais **contratos**. Essas interfaces representam o **modelo de domínio**, os **dados de formulário**, os **dados vindos da API** e as **estruturas usadas pelas UI (Componentes)**.

Elas ficam centralizadas em `Interfaces/Interfaces.ts` e servem como:

- Contrato de exibição do frontend para facilitar o desenvolvimento da aplicação.
- Fonte de verdade para tipagem.
- Base para validações e mapeamentos.
- Documentação viva do domínio de Ativos.



Contratos Base

Schema

Contrato padrão usado pelos schemas de formulário.

```
export interface Schema { label: string; value: string }[]
```

FetchConfig

Contrato padrão para configuração de buscas na API.

```
export interface FetchConfig {  
  pathFile: string;
```

```
urlComplement?: string;
params?: {};
}
```

Conjunto de Dados da Tela

ActiveTableData

Representa o pacote completo de dados que a tela de Ativos precisa para funcionar.

Inclui:

- Dados do ativo
- Dados do veículo (se aplicável)
- Listas de apoio (motoristas, empresas, unidades, tipos, etc)

```
export interface ActiveTableData {
  active: ActiveFormValues;
  vehicle: VehicleFormValues;
  driver: Driver[];
  company: Company[];
  unit: Unit[];
  activeType: ActiveType[];
  fuelType: FuelType[];
  departament: Departament[];
}
```

Endereços

PlaceAddress

Representa o endereço onde o ativo foi comprado ou adquirido. É flexível, pois pode estar incompleto durante o cadastro.

```
export interface PlaceAddress {
  city?: string;
  state?: string;
  store?: string;
  number?: string;
  zip_code?: string;
  complement?: string;
  neighborhood?: string;
  public_place?: string;
}
```

Estrutura Organizacional

Departament

Representa um departamento da empresa dentro da estrutura organizacional.

Na regra de negócio, serve para:

- Organizar onde o ativo está alocado
- Relacionar ativo com unidade e empresa

```
export interface Departament {
  dep_id: number;
  dep_name: string;
  status_dep: number;
  unit_id_fk: number;
  unit_id: number;
  unit_number: number;
  address: string;
  unit_name: string;
  cnpj: string;
  status_unit: number;
  comp_id_fk: number;
  comp_id: number;
  corporate_name: string;
  fantasy_name: string;
  status_comp: number;
}
```

Listas Dinâmicas

IListAdd

Representa as ações de negócio para gerenciar a lista de itens de um ativo (ex: acessórios, componentes, itens vinculados).

```
export interface IListAdd {
  newItemText: string;
  setNewItemText: React.Dispatch<React.SetStateAction<string>>;
  addItem: () => void;
  activeValues: ActiveFormValues;
  removeItem: (indexToRemove: number) => void;
}
```

Formulário de Ativo

ActiveFormValues

Representa os dados do Ativo enquanto ele está sendo cadastrado ou editado. É o “rascunho” do ativo antes de ser salvo.

```
export interface ActiveFormValues {
  active_id?: number | null;
  brand?: string;
  model?: string;
  number_nf?: number | string;
  date_purchase?: string;
  place_purchase?: PlaceAddress;
  value_purchase?: number | string;
}
```

```

photo?: File | string | null;
change_date?: string | null;
list_items?: {
  list: string[];
};
used_in?: number | string | null;
is_vehicle?: boolean;
status_active?: number | string;
units_id_fk?: number | string | null;
id_active_class_fk?: number | string | null;
user_id_fk?: number | string | null;
work_group_fk?: number | string | null;
}

```

VehicleFormValues

Representa os dados específicos quando o ativo é um veículo.

```

export interface VehicleFormValues {
  license_plates?: string;
  year?: number | string;
  year_model?: number | string;
  chassi?: string;
  color?: string;
  renavam?: string;
  fuel_type?: string;
  power?: number | string;
  cylinder?: number | string;
  capacity?: number | string;
  fiipe_table?: number | string;
  last_revision_date?: string | null;
  last_revision_km?: number | string;
  next_revision_date?: string | null;
  next_revision_km?: number | string;
  directed_by?: number | string | null;
  shielding?: boolean;
  fuel_type_id_fk?: number | string | null;
}

```



Entidades do Domínio

Company

```

export interface Company {
  comp_id: number;
  corporate_name: string;
  fantasy_name: string;
  status_comp: number;
}

```

ActiveType

Define a classificação do ativo (ex: veículo, equipamento, máquina, etc).

```

export interface ActiveType {
  active_type_id: number;
}

```



```
desc_active_type: string;
date_active_type: string;
status_active_type: number;
group_id_fk: string;
}
```

FuelType

```
export interface FuelType {
  id_fuel_type: number;
  description: string;
}
```

Unit

Representa uma unidade/filial da empresa.

```
export interface Unit {
  unit_id: number;
  unit_number: number;
  address: AddressUnit;
  unit_name: string;
  cnpj: string;
  status_unit: number;
  comp_id_fk: number;
  comp_id: number;
  corporate_name: string;
  fantasy_name: string;
  status_comp: number;
}
```

Driver

Representa um motorista ou usuário responsável por veículos.

```
export interface Driver {
  driver_id: number;
  rg: string;
  cpf: string;
  cnh: string;
  category: string;
  validity_cnh: string;
  status_driver: number;
  cnh_img?: string;
  user_id_fk: string;
  sub_dep_id_fk: string;
  level_id_fk: string;
  user_id: string;
  name: string;
  access_code: string;
  driver: string;
  status_user: string;
  level_id: string;
  level_name: string;
  level_pages: {
    level: string[];
  };
  status_level: string;
}
```

```
    group_id_fk: string;
}
```



Entidade Principal

Active

Representa o Ativo já cadastrado no sistema, com todas as informações completas. É o registro oficial usado para listagem, consulta e relatórios.

```
export interface Active extends ActiveRecord {
  active_id: string;
  brand: string;
  model: string;
  number_nf: string;
  status_active: string;
  photo?: null;
  change_date: string;
  list_items: {
    list: string[];
  };
  used_in: string;
  date_purchase: string;
  place_purchase: PlaceAddress;
  value_purchase: string;
  is_vehicle: number;
  units_id_fk: string;
  id_active_class_fk: string;
  user_id_fk: string;
  work_group_fk: string;
  unit_id: number;
  unit_number: number;
  address: string;
  unit_name: string;
  cnpj: string;
  status_unit: number;
  comp_id_fk: number;
  id_active_class: string;
  desc_active_class: string;
  status_active_class: string;
  active_type_id_fk: string;
  user_id: string;
  name: string;
  access_code: string;
  driver: string;
  status_user: string;
  sub_dep_id_fk: string;
  level_id_fk: string;
  group_id: number;
  group_name: string;
  status_work_group: number;
  dep_id_fk: number;
}
```



Props do Formulário

FormActiveProps

Define os dados que o formulário de Ativo pode receber, permitindo:

- Abrir em modo edição
- Controlar abertura/fechamento de modal
- Injetar dados vindos da API

```
export interface FormActiveProps {  
  apiData?: {  
    active?: ActiveFormValues;  
    departament?: Departament[];  
    vehicle?: VehicleFormValues;  
    driver?: Driver[];  
    company?: Company[];  
    unit?: Unit[];  
    activeType?: ActiveType[];  
    fuelType?: FuelType[];  
  };  
  openModal?: Dispatch<SetStateAction<boolean>>;  
}
```